



Graduação Presencial Síncrona ...

Página inicial

Teams

Avisos

Tarefas

Fóruns

Biblioteca PUC Minas

Notas

6

Pessoas

Páginas

Arquivos

Programa

Módulos

Colaborações

Caderno

Office 365

Lucid (Quadro branco)

Avaliação CPA

PUC Carreiras

Visualizar todas as páginas

Laboratório 02 — Assistentes de IA vs. Codificação Manual

LAB02 — Laboratório 02 · 20 pontos

Curso	Disciplina
Engenharia de Software	Laboratório de Experimentação de Software
Turno	Período / Sala
Noite	6º
Professor(a)	
Danilo Maia	

Assistentes de IA vs. codificação manual: um experimento controlado

Ferramentas de IA generativa (GitHub Copilot, ChatGPT, Claude, Gemini, etc.) tornaram-se onipresentes no desenvolvimento de software, mas ainda há pouca evidência controlada e reproduzível sobre seu real impacto em produtividade e qualidade — a maior parte do que se ouve é relato anedótico. Neste laboratório, o objetivo é realizar um **experimento controlado** para avaliar quantitativamente os efeitos do uso de um assistente de IA na resolução de tarefas de programação.

Questões de Pesquisa

- RQ1** O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?
- RQ2** O uso de assistente de IA reduz a quantidade de defeitos (testes que falham) no código produzido?
- RQ3** O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código produzido? (métricas via CK — apenas Java — e/ou PMD; use uma ferramenta equivalente, como Radon, se a linguagem escolhida não for Java)

GQM (Goal-Question-Metric) e Métricas

Goal

Analisar o uso de assistentes de IA generativa na resolução de tarefas de programação, com o propósito de comparar seu efeito frente à codificação manual, com respeito a tempo de resolução, qualidade funcional (defeitos) e qualidade estrutural do código produzido, do ponto de vista do grupo pesquisador, no contexto de katas de dificuldade equivalente resolvidos por estudantes de graduação sob condições controladas (crossover within-subject, time-boxed).

As RQ1–RQ3 já definidas acima **são** as Questions do GQM. Cabe a cada grupo escolher, entre as métricas candidatas abaixo, quais usar para responder cada RQ — a escolha e a justificativa devem constar no Desenho do Experimento (Passo 1) e no Relatório Final.

- RQ1** Tempo — métricas candidatas
- Tempo até passar em todos os testes de aceitação ("time-to-green") — métrica primária recomendada.
 - Trial que atinge o time-box (35 min) sem sucesso deve ser registrado como **censurado em 35 min**, não descartado — descartar distorce a comparação a favor do tratamento com mais falhas.
 - Métrica agregada recomendada: **mediana** por tratamento (não a média), dado o N pequeno (4–6 trials/integrante) e a sensibilidade da média a outliers.
 - Opcional/exploratória: n° de prompts/interações com o assistente de IA — não obrigatória, mas útil para discussão qualitativa.

- RQ2** Defeitos — métricas candidatas
- Taxa de sucesso: % de testes de aceitação passando ao final do time-box — mais robusta que a contagem bruta, pois normaliza katas com números diferentes de testes.
 - N° absoluto de testes falhando ao final do tempo — métrica complementar, mais simples de reportar.
 - Opcional: densidade de defeitos (testes falhando / KLOC), se quiserem comparar katas de tamanhos bem diferentes.

RQ3 Estrutura do código — métricas candidatas

- *Complexidade ciclomática média* (McCabe) por método/função — via CK (Java, métrica WMC/complexity) ou Radon `cc` (Python).
- *Duplicação de código*: % de linhas duplicadas via PMD CPD (Java) ou ferramenta equivalente (ex.: `jscpd` para Python/JS, se Radon não cobrir duplicação).
- **LOC (linhas de código) como métrica de controle** — obrigatória sempre que reportar complexidade/duplicação: código gerado por IA pode ser mais verboso, e complexidade/duplicação sem normalizar por LOC pode enganar.
- *Opcional (aprofundamento)*: Índice de Manutenibilidade (Maintainability Index, disponível no Radon `mi`) — métrica composta (complexidade + LOC + volume de Halstead), mais robusta que olhar cada métrica isoladamente.

Robustez estatística: dado o tamanho amostral reduzido, prefira **mediana e IQR** (intervalo interquartil) a média e desvio-padrão nas tabelas e gráficos descritivos, e mantenha o teste de Wilcoxon (não paramétrico) na análise inferencial do Passo 4 — consistente com o desenho within-subject.

Etapas esperadas por sprint

1 Desenho do Experimento

Defina, no mínimo: (A) Hipóteses nula e alternativa; (B) Variáveis dependentes (tempo, nº de testes passando, métricas estáticas); (C) Variável independente (uso ou não do assistente de IA); (D) Tratamentos; (E) Objetos experimentais (conjunto de exercícios/katas de dificuldade equivalente); (F) Tipo de projeto experimental (recomenda-se **crossover/within-subject**, contrabalanceado, para controlar variação individual de habilidade); (G) Quantidade de medições; (H) Ameaças à validade (efeito de aprendizado entre katas, familiaridade prévia com a ferramenta de IA, vazamento de solução já vista, **e memorização**: se as katas forem muito conhecidas — ex.: exercícios clássicos do LeetCode/HackerRank —, o assistente de IA pode reproduzir uma solução já vista em seu treinamento em vez de efetivamente "ajudar"; para reduzir esse risco, prefira katas autorais do grupo/professor ou exercícios pouco indexados).

2 Preparação do Experimento

Escolha de **4 ou 6 katas**/exercícios de programação de dificuldade comparável — número par, para permitir a divisão exata pela metade entre trials com e sem assistente de IA (ex.: HackerRank, LeetCode, Codewars, ou exercícios próprios — prefira exercícios pouco indexados para reduzir o risco de memorização, ver ameaça à validade acima) —, com testes automatizados de aceitação. Prepare o ambiente (linguagem, IDE, assistente de IA a ser usado, cronômetro/registro de tempo, scripts de coleta das métricas estáticas). **O grupo deve usar o mesmo assistente de IA em todos os trials**, para que o tratamento seja comparável dentro do próprio experimento (ex.: GitHub Copilot gratuito via GitHub Student Developer Pack, ou a versão gratuita de um chatbot como ChatGPT/Claude/Gemini). Fixe também a linguagem de programação das katas de acordo com a ferramenta de métricas estáticas escolhida (CK exige Java; para outras linguagens, use uma ferramenta equivalente, como Radon para Python).

3 Execução do Experimento

Cada integrante do grupo resolve metade dos katas com assistente de IA habilitado e a outra metade sem, em ordem contrabalanceada entre os integrantes, sob tempo limitado: **35 minutos por trial** (o grupo pode reduzir esse limite e justificar no relatório, mas não pode aumentá-lo, para manter a comparabilidade entre grupos da turma). Ao final do tempo, o trial é encerrado independentemente do resultado. Registre: tempo até passar nos testes de aceitação (ou até o fim do time-box), nº de testes passando ao final do tempo, e execute CK/PMD sobre o código final de cada trial.

Time-box fixo: **35 min/trial** — só pode ser reduzido, nunca aumentado.

4 Análise de Resultados

Revise os dados coletados, identifique outliers, e aplique os testes estatísticos adequados (ex.: teste de Wilcoxon para amostras pareadas, dado o desenho within-subject).

5 Relatório Final

Documento com: (i) introdução com as hipóteses; (ii) metodologia detalhada o suficiente para permitir reprodução/replicação (ambiente, katas usados, assistente de IA e versão); (iii) resultados por RQ com as respostas estatísticas obtidas; (iv) discussão final; (v) o link do repositório/GitHub Projects do grupo.

Link do repositório/GitHub Projects: `<preencher>`

6 Dashboard de Visualização

Importe os dados do experimento e gere gráficos (Pandas + Matplotlib/Seaborn) comparando tempo, taxa de sucesso e métricas estáticas entre os tratamentos.

Processo de Desenvolvimento

Contribuição individual por sprint. Em toda sprint (S01, S02 e S03), cada integrante do trio deve ser Assignee de ao menos uma Issue com artefato de código commitado (script, notebook, gráfico ou trial de kata) — não apenas nas Issues de execução de katas da S02. A ausência de commits atribuíveis a um integrante em uma sprint zera a parcela individual

daquele integrante na sprint.

- Sugestão de divisão de papéis por sprint** — não obrigatória: o trio é livre para se organizar de outra forma, desde que a regra acima seja respeitada.
- S01: um integrante escreve o script de cronometragem/coleta de tempo; outro prepara o ambiente e o script de execução das métricas estáticas (CK/PMD ou Radon); o terceiro pesquisa e valida os katas (dificuldade comparável, baixa indexação) e redige hipóteses e ameaças à validade — os três revisam o desenho em conjunto.
 - S02: já naturalmente dividida por design — cada integrante resolve, individualmente, todos os katas (metade com IA, metade sem), em ordem contrabalanceada.
 - S03: um integrante conduz os testes estatísticos (Wilcoxon) para RQ1/RQ2; outro conduz a análise da RQ3 (métricas estáticas); o terceiro monta o dashboard (Pandas/Matplotlib/Seaborn) consolidando os resultados dos três.

Sprint	Entregável	Pontos
Lab02S01	Desenho do experimento + preparação (Passos 1-2: katas escolhidos, ambiente, scripts de medição de tempo e métricas). Cartões do desenho e da preparação devem estar no Kanban do grupo.	5
Lab02S02	Execução do experimento + coleta de dados (Passo 3).	5
Lab02S03	Análise de resultados (Passo 4, cobrindo RQ1, RQ2 e RQ3) + Dashboard de Visualização (Passo 6).	5
Relatório Final	Elaboração do documento final (Passo 5 — ver seção "Relatório Final" acima).	5
Total		20

Prazo final: conforme cronograma da disciplina.

Desconto de até 10% da nota da sprint por qualidade insuficiente do uso do GitHub Projects (WIP não respeitado, Issues sem Assignee, cartões desatualizados, ausência de evolução semanal).

Observação: todos os trials devem ser registrados no GitHub Projects do grupo como Issues individuais (uma por kata/tratamento), atribuídas ao integrante responsável (campo Assignee), mantendo a rastreabilidade entre o experimento e o board. A correção é feita a partir do GitHub Projects: commits sem referência ao número da Issue correspondente não serão considerados.

◀ Anterior

Próximo ▶